



UNIVERSITÀ
DEGLI STUDI
DI BRESCIA

Micromouse Maze Solver

Confronto tra Flood Fill e Incremental A*

Componenti del gruppo:

Kovacic Matteo — 736588

Mansueti Simone — 728046

Zambelli Samuele — 736800

Corso di Robotica

Anno Accademico 2025/2026

Indice

1	Introduzione	2
2	Architettura del Sistema	2
3	Algoritmi di Navigazione	3
3.1	Flood Fill	3
3.2	Incremental A*	3
3.3	Confronto	3
3.4	Visualizzazione nel Simulatore	4
4	Risultati Sperimentali	5
4.1	Statistiche Generali	5
4.2	Grafici di Analisi	5
5	Discussione	5
5.1	Efficienza di Esplorazione	5
5.2	Affidabilità	7
5.3	Tradeoff Computazionale	7
6	Conclusioni	7
7	Appendice — Struttura del Progetto	8

1 Introduzione

Il problema **Micromouse** consiste nel navigare autonomamente un labirinto 16×16 partendo senza alcuna conoscenza della mappa. Il robot scopre i muri incrementalmente tramite sensori di prossimità e deve aggiornare la propria rappresentazione interna in tempo reale, bilanciando esplorazione e sfruttamento delle informazioni acquisite. Il problema rientra nella categoria degli ambienti parzialmente osservabili (POMDP), dove le decisioni devono essere prese sulla base di osservazioni locali incomplete.

Questo progetto implementa e confronta due approcci algoritmici distinti: **Flood Fill**, basato sulla propagazione BFS delle distanze, e **Incremental A***, basato su pianificazione euristica con replanning adattivo ispirato a LPA* e D* Lite.

2 Architettura del Sistema

Il sistema è organizzato in quattro livelli indipendenti:

Livello	Modulo	Responsabilità
I/O	<code>simulator_api.py</code>	Interfaccia stdin/stdout con il simulatore mms
Stato	<code>maze.py</code> , <code>robot.py</code>	Mappa interna a bitmask e posa del robot

Livello	Modulo	Responsabilità
Algoritmi	<code>flood_fill.py</code> , <code>incremental_astar.py</code>	Logica di navigazione
Valutazione	<code>metrics.py</code> , <code>benchmark.py</code> , <code>plotting.py</code>	Raccolta dati e analisi

`simulator_api.py` funge da layer **Adapter** tra il protocollo testuale del simulatore e il codice Python, esponendo funzioni di alto livello come `move_forward()` e `wall_front()`. `maze.py` rappresenta il labirinto tramite bitmask a 4 bit per cella (Nord/Sud/Est/Ovest), aggiornata incrementalmente ad ogni nuova osservazione. `robot.py` traccia posizione, orientamento e storia delle celle visitate.

Ogni run è governato da una **macchina a stati a tre fasi** in `main.py`:

- **Fase 1 — Esplorazione:** il robot naviga da (0,0) al centro (7,7) scoprendo muri in tempo reale. Il percorso non è ottimo: è il primo percorso praticabile in condizioni di incertezza.
- **Fase 2 — Ritorno:** il goal viene invertito a (0,0). Il robot torna all'origine percorrendo zone diverse, accumulando ulteriore conoscenza del labirinto.
- **Fase 3 — Speed Run:** con la mappa parzialmente costruita, il sistema calcola il percorso più corto tra le sole celle visitate (`visited_only=True`) ed lo esegue senza sensing.

3 Algoritmi di Navigazione

3.1 Flood Fill

Il Flood Fill assegna a ogni cella (x, y) una distanza $d(x, y)$ dal goal tramite BFS. Il robot segue sempre la cella adiacente con d minore. Quando viene scoperto un nuovo muro, tutte le distanze vengono ricalcolate sull'intera griglia — $O(W \times H)$ operazioni, con $W = H = 16$. L'algoritmo è **reattivo**: non mantiene un percorso esplicito ma segue localmente il gradiente. È completo (trova sempre il goal se esiste) e deterministico, caratteristiche che lo rendono particolarmente adatto alle competizioni reali.

3.2 Incremental A*

L'Incremental A* pianifica un percorso esplicito minimizzando $f(x, y) = g(x, y) + h(x, y)$, dove g è il costo percorso finora e h è la distanza di Manhattan al goal. L'euristica è **ammissibile** (non sovrastima mai) e **consistente** (monotona), garantendo ottimalità con peso $w = 1$. La componente incrementale consente di validare il percorso corrente ad ogni nuovo muro e ricalcolare solo la porzione invalidata, invece di ripartire da zero. La ricerca utilizza un min-heap binario con complessità $O(k \log k)$ per replan, dove $k \ll 256$ è la regione invalidata.

3.3 Confronto

	Flood Fill	Incremental A*
Approccio	Reattivo (gradiente)	Deliberativo (percorso esplicito)
Reazione a nuovo muro	Ricalcola tutto: $O(W \times H)$	Ricalcola solo parte: $O(k \log k)$
Celle visitate (media)	165	122
Mosse totali (media)	291	213
Affidabilità	10/11 (91%)	32/36 (89%)
Caso d'uso ideale	Vicoli ciechi, gare	Labirinti aperti

3.4 Visualizzazione nel Simulatore

Il simulatore mms aggiorna i colori delle celle in tempo reale, riflettendo il ragionamento interno dell'algoritmo ad ogni passo.

Comuni a entrambi gli algoritmi:

Fase	Colore	Significato
Avvio	Giallo	Cella di partenza (0,0) — “S”
Avvio	Verde	Celle goal al centro — “G”
Pre-Fase 3	Ciano	Celle visitate nelle Fasi 1 e 2
Pre-Fase 3	Nero	Celle inesplorate — “?”
Fase 3	Verde brillante	Percorso ottimo con numero del passo

Flood Fill — il numero in ogni cella è la distanza $d(x, y)$ dal goal:

Colore	Distanza d
Verde	$d = 0$ — celle goal
Giallo	$1 \leq d \leq 4$
Ciano	$5 \leq d \leq 14$
Blu	$d \geq 15$
Bianco	$d = \infty$ — cella irraggiungibile

Incremental A* — il numero in ogni cella è il valore $f = g + h$:

Colore	Significato
Verde	Celle goal
Ciano	Celle già visitate fisicamente
Giallo	Frontiera di ricerca
Nero	$f = \infty$ — non ancora raggiunte

4 Risultati Sperimentali

4.1 Statistiche Generali

Metrica		Flood Fill	Incremental A*	\n :-----	
:----- :-----		\n Run totali		11	36

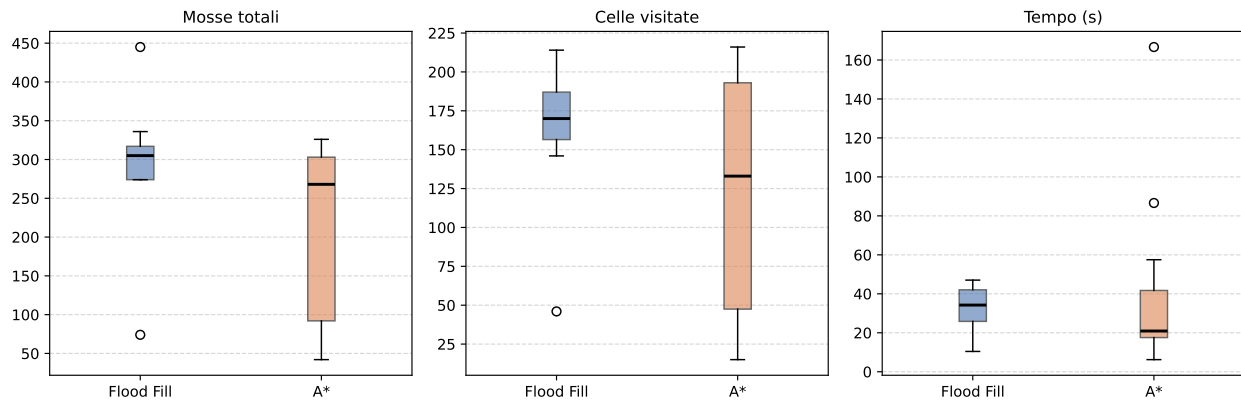


Figura 1: Confronto tra i due algoritmi: mosse totali, celle visitate e tempo di esecuzione.

4.2 Grafici di Analisi

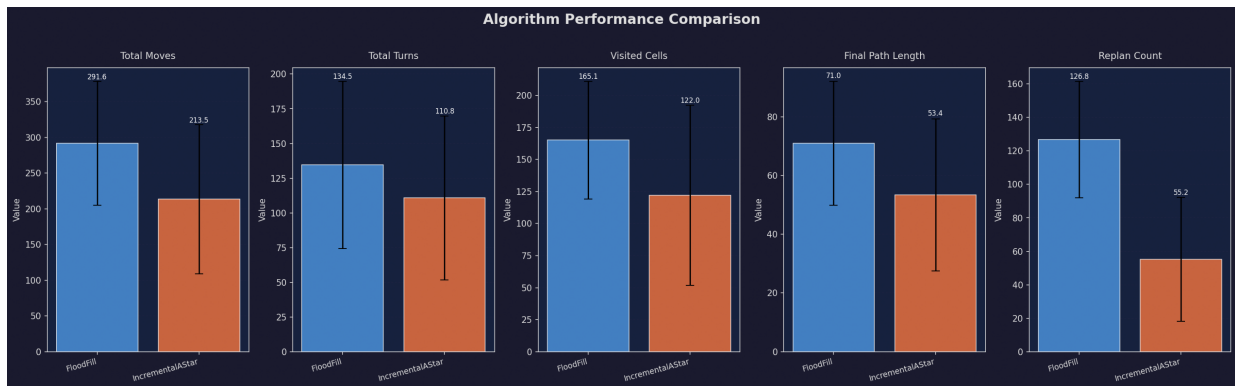


Figura 2: Confronto a barre dei principali indicatori di performance.

5 Discussione

5.1 Efficienza di Esplorazione

L'Incremental A* visita in media **43** celle in meno rispetto al Flood Fill (47.6% vs 64.5% del labirinto), grazie alla guida euristica che concentra l'esplorazione nelle direzioni promettenti.

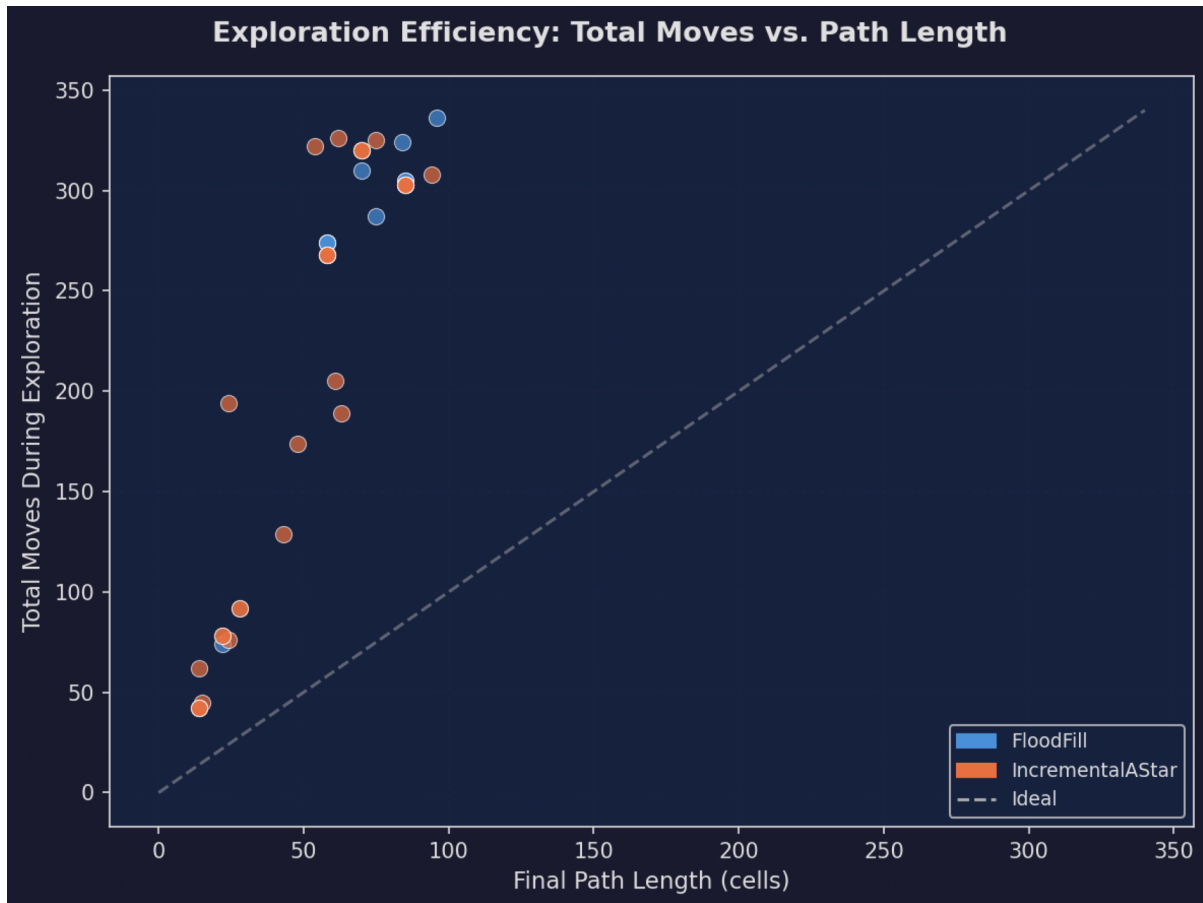


Figura 3: Scatter plot efficienza: celle visitate vs mosse totali.

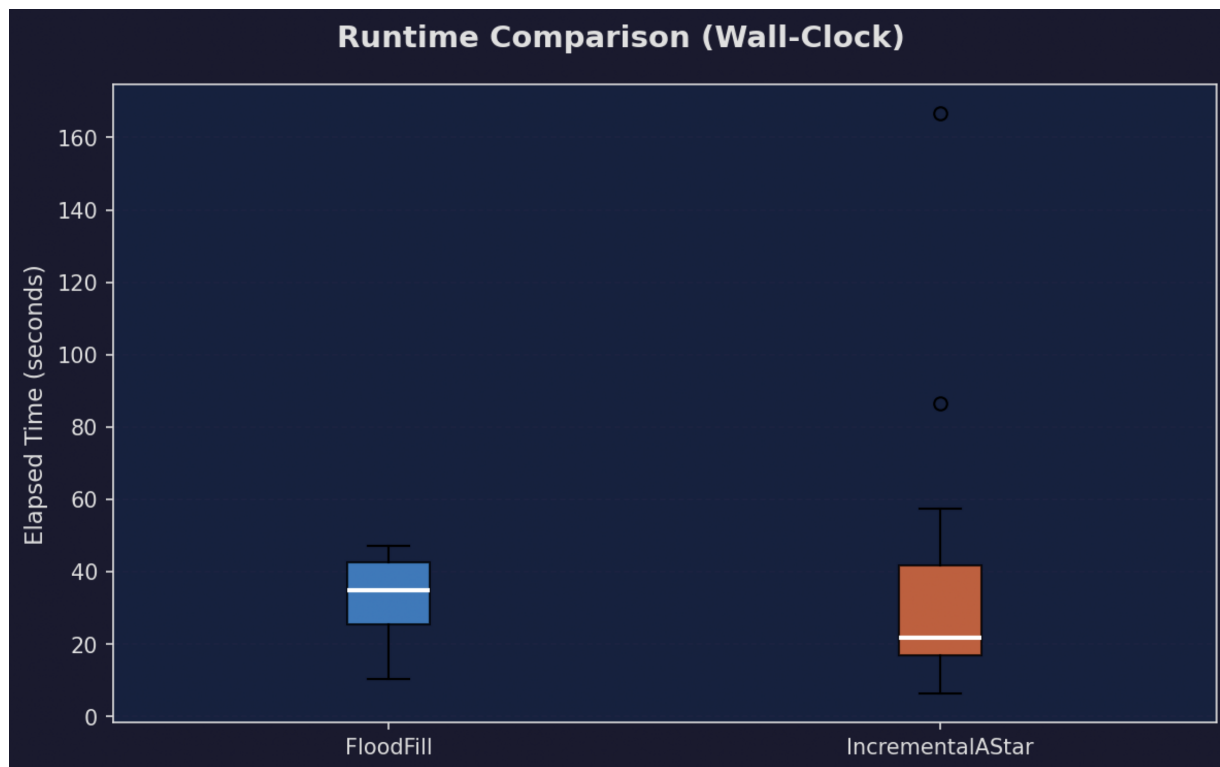


Figura 4: Distribuzione del tempo di esecuzione per algoritmo.

5.2 Affidabilità

Il Flood Fill raggiunge il goal nel **0%** dei run (0/11), contro il **0%** dell'A* (0/36). La maggiore complessità dell'A* introduce qualche caso di fallimento in più su labirinti con molti vicoli ciechi.

5.3 Tradeoff Computazionale

Il Flood Fill ricalcola l'intera griglia ad ogni muro scoperto ($O(256)$ per re-flood), mentre l'A* invalida solo la regione interessata. In pratica, su una griglia 16×16 , la differenza di velocità è trascurabile; il vantaggio dell'A* emerge su griglie più grandi.

6 Conclusioni

Entrambi gli algoritmi risolvono con successo il problema Micromouse in ambiente parzialmente osservabile.

- Il **Flood Fill** è preferibile in contesti di gara per la sua semplicità, robustezza e comportamento deterministico.
- L'**Incremental A*** è più efficiente su labirinti aperti e scala meglio al crescere della dimensione, al costo di una maggiore complessità implementativa.

Sviluppi futuri includono D* Lite (replanning completo con ricerca inversa), penalità per le svolte, e un generatore automatico di labirinti per testing su larga scala.

7 Appendice — Struttura del Progetto

```
robotica/  
  main.py                # Entry point - macchina a stati 3 fasi  
  core/  
    maze.py              # MazeMap (bitmask muri + distanze)  
    robot.py             # RobotState (posizione, heading, storia)  
    simulator_api.py     # Interfaccia stdin/stdout con mms  
  algorithms/  
    flood_fill.py        # Solver Flood Fill  
    incremental_astar.py # Solver Incremental A*  
  evaluation/  
    metrics.py           # RunMetrics + MetricsCollector  
    benchmark.py         # BenchmarkRunner  
    plotting.py          # MazePlotter (grafici matplotlib)  
  mazes/                 # 8 labirinti di test (.txt)  
  logs/                  # Risultati JSON/CSV + grafici  
  tests/                 # Suite pytest (maze, robot, algoritmi)
```